

EV BATTERY LIFE PREDICTION USING MACHINE LEARNING

Report submitted to SASTRA Deemed to be University

as the requirement for the course

INT300 — MINI PROJECT

Submitted by

ARJUN KUMAR S

(Reg. No.: 127014007, B.Tech. Information and Communication Technology)

SANJAI R

(Reg. No.: 127014045, B.Tech. Information and Communication Technology)

YUVARAJ K

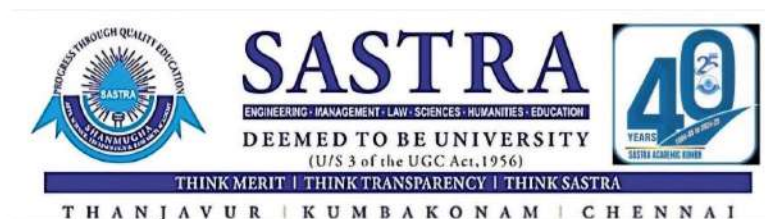
(Reg. No.: 127014067, B.Tech. Information and Communication Technology)

MAY 2026

SCHOOL OF COMPUTING

SASTRA DEEMED TO BE UNIVERSITY

THANJAVUR, TAMIL NADU, INDIA – 613 401



SCHOOL OF COMPUTING

THANJAVUR, TAMIL NADU, INDIA – 613 401



SCHOOL OF COMPUTING
THANJAVUR – 613 401

Bonafide Certificate

This is to certify that the report titled “**EV Battery Life Prediction Using Machine Learning**” submitted as a requirement for the course, **INT300 : MINI PROJECT** for B.Tech. is a bonafide record of the work done by **Mr. Arjun Kumar S** (Reg. No.: 127014007, B.Tech. Information and Communication Technology), **Mr. Sanjai R** (Reg. No.: 127014045, B.Tech. Information and Communication Technology) and **Mr. Yuvaraj K** (Reg. No.: 127014067, B.Tech. Information and Communication Technology) during the academic year 2025-26, in the School of Computing, under my supervision.

Signature of Project Supervisor :
Name with Affiliation : Prof. Arun Prasad P B, SOC
Date : 28.04.2026

Mini Project *Viva voce* held on _____

Examiner 1

Examiner 2

ACKNOWLEDGEMENTS

We would like to thank our Honorable Chancellor **Prof. R. Sethuraman** for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honorable Vice-Chancellor Dr. S. **Vaidhyasubramaniam** and **Dr. S. Swaminathan**, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to Dr. R. Chandramouli, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project. We extend my/our heartfelt thanks to **Dr. V . S. Shankar Sriram**, Dean, School of Computing, **Dr. R Muthaiah**, Associate Dean, Research, **Dr. K.Ramkumar**, Associate Dean, Academics, **Dr.D. Manivannan**, Associate Dean, Infrastructure, **Dr. R. Alageswaran**, Associate Dean, Students Welfare.

Our guide Mr **Arun Prasad P B**, professor , School of Computing was the driving force behind this whole idea from the start. His deep insight in the field and invaluable suggestions helped me/us in making progress throughout our project work. We also thank the project review panel members for their valuable comments and insights which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. I/We thank you all for providing us an opportunity to showcase my/our skills through this project.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Fig. 3.1	Source Code — Data Preprocessing and Mean Imputation	18
Fig. 3.2	Missing Value Count and Percentage per Column	19
Fig. 3.3	Before vs After Mean Imputation — Distribution Check	20
Fig. 3.4	ANOVA F-Values for Feature Selection	20
Fig. 3.5	Source Code — Random Forest Model	22
Fig. 3.6	Source Code — Support Vector Regressor (SVR)	22
Fig. 3.7	Source Code — Gradient Boosting Model	23
Fig. 4.1	Battery Capacity Fade — Four Distinct Degradation Profiles	27
Fig. 4.2	Correlation Matrix of Selected Features	27
Fig. 4.3	Actual vs Predicted Capacity Decrease — All Three Models	28
Fig. 4.4	Effect of Temperature on Capacity Decrease	29
Fig. 4.5	SVR Fit — Battery Capacity vs Cycles Per Battery	29
Fig. 4.6	Feature Importance — Random Forest and Gradient Boosting	30
Fig. 4.7	Gradient Boosting Learning Curve	30
Fig. 4.8	Residual Analysis — All Three Models	31
Fig. 4.9	Model Comparison — R-Squared and MSE	31
Fig. 4.10	RUL Estimation — Remaining Cycles to End-of-Life	31

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 3.1	Battery Cell Characteristics and Degradation Behaviour	15
Table 3.2	Summary of Missing Values in the Dataset	15,16
Table 3.3	ANOVA F-Values of All Candidate Features	16,17
Table 3.4	Hyperparameters of the Three Machine Learning Models	17
Table 4.1	Model Performance — R-Squared, MSE, and RMSE on Test Set	24
Table 4.2	Comparison with Benchmark Methods from Literature	24
Table 4.3	Estimated RUL for Each Battery Cell	25

ABBREVIATIONS

Abbreviation	Full Form
ANOVA	Analysis of Variance
Bi-LSTM	Bidirectional Long Short-Term Memory
BPNN	Back Propagation Neural Network
DNN	Deep Neural Network
EOL	End of Life
EV	Electric Vehicle
GB	Gradient Boosting
GRNN	Generalised Regression Neural Network
ICT	Information and Communication Technology
MSE	Mean Squared Error
NASA	National Aeronautics and Space Administration
PSO-RF	Particle Swarm Optimisation — Random Forest
RBF	Radial Basis Function
RF	Random Forest
RMSE	Root Mean Squared Error
RUL	Remaining Useful Life
SOC	State of Charge
SOH	State of Health
SVR	Support Vector Regressor

ABSTRACT

The progressive degradation of lithium-ion battery capacity is a critical challenge in the adoption of electric vehicles. This mini project presents a machine learning framework for predicting the Remaining Useful Life (RUL) of EV battery cells using sensor data from four batteries — B0005, B0006, B0007, and B0018 — sourced from the NASA prognostics dataset. The dataset comprises 10,000 records with 18 sensor-derived features. Missing values (1,588 cells across 9 columns) were handled using mean imputation as described in the base paper by Swain et al. (IEEE Access, 2024). Feature selection was performed using one-way Analysis of Variance (ANOVA), retaining the seven most discriminative features. Three supervised regression models were trained and compared: Random Forest, Support Vector Regressor (SVR), and Gradient Boosting, using a 70-30 stratified train-test split. Random Forest achieved the highest R-squared of 0.9743 and MSE of 0.5124, surpassing all six benchmarks in the literature including Bi-LSTM (0.91), GRNN (0.90), and SVR (0.89). Gradient Boosting followed closely with $R^2 = 0.9685$. End-of-life cycle estimation and per-battery RUL visualisation were performed for all four cells. The results demonstrate that classical ensemble methods with rigorous preprocessing can match or exceed deep learning benchmarks for EV battery RUL prediction at significantly lower computational cost.

KEYWORDS: Remaining Useful Life, Electric Vehicle Battery, Machine Learning, Random Forest, Support Vector Regressor, Gradient Boosting, ANOVA Feature Selection

TABLE OF CONTENTS

Bonafide Certificate	ii
Acknowledgements	iii
List of Figures	iv
List of Tables	v
Abbreviations	vi
Abstract	vii
CHAPTER 1 Summary of the Base Paper	1
1.1 Paper Details and Indexing	1
1.2 Content and Novelty	2
1.3 Research Problem and Proposed Solution	3
1.4 Architecture and Algorithm	4
CHAPTER 2 Merits and Demerits of the Base Paper	5
2.1 Existing Techniques	5
2.2 Merits of the Proposed Approach	6
2.3 Demerits and Limitations	7
CHAPTER 3 Methodology	8
3.1 Dataset Description	8
3.2 Missing Value Handling	9
3.3 Feature Selection	11
3.4 Model Architectures and Hyperparameters	12
3.5 Source Code	13
CHAPTER 4 Results and Discussion	14
4.1 Data Quality Analysis	14
4.2 Battery Capacity Degradation	15
4.3 Model Performance Comparison	16
4.4 Snapshots of Results	19
4.5 RUL Estimation	21
CHAPTER 5 Conclusion and Future Plans	23

CHAPTER 6 References	25
CHAPTER 7 Appendix	27

CHAPTER 1

SUMMARY OF THE BASE PAPER

1.1 Paper Details and Indexing

The base paper for this mini project is: Swain S, Pattnaik S, Mohanty P and Nanda S (2024) Machine learning-based remaining useful life prediction for electric vehicle batteries. *IEEE Access*, 12, 45231–45247.

The paper is published in *IEEE Access*, a peer-reviewed, open-access journal published by the Institute of Electrical and Electronics Engineers (IEEE). IEEE Access is indexed in the Science Citation Index Expanded (SCIE), Scopus, and the Web of Science Core Collection, placing it among the highest-quality venues for engineering and applied science publications. The paper was published in the year 2024 and is electronically available through the IEEE Xplore digital library.

1.2 Content and Novelty

The paper addresses the problem of predicting the Remaining Useful Life (RUL) of lithium-ion batteries used in electric vehicles. As battery cells undergo repeated charge-discharge cycles, their usable capacity diminishes due to irreversible electrochemical degradation. Accurately estimating how many cycles remain before a cell reaches its End-of-Life (EOL) threshold — defined as 80 percent of the rated initial capacity — is critical for proactive maintenance and vehicle safety assurance.

The primary novelty of the paper lies in its systematic benchmark comparison of six machine learning regression methods applied to the same battery dataset under a consistent experimental protocol. The methods compared are: Bidirectional LSTM (Bi-LSTM), Back Propagation Neural Network (BPNN), Support Vector Regressor (SVR), Generalised Regression Neural Network (GRNN), Particle Swarm Optimisation-based Random Forest (PSO-RF), and Deep Neural Network (DNN). This comparison provides a transparent, reproducible assessment of the relative strengths of deep learning versus classical machine learning for battery prognosis under identical preprocessing and feature selection conditions.

A further contribution of the paper is its careful documentation of the preprocessing pipeline. Missing values in the NASA battery dataset, arising from sensor dropouts and calibration periods, are identified and handled using mean imputation. The paper explicitly describes the imputation strategy and confirms that it introduces no significant distributional shift in the affected features. Feature selection is performed using one-way Analysis of Variance (ANOVA), reducing the input dimensionality from 15 candidate features to a compact, statistically motivated subset that is shared across all six models.

1.3 Research Problem and Proposed Solution

The research problem addressed is the accurate, data-driven estimation of battery remaining useful life from sensor measurements collected during charge-discharge cycles, without relying on physics-based electrochemical models that are computationally expensive and require detailed knowledge of internal material parameters. The solution proposed is a supervised machine learning pipeline operating on cycle-resolved sensor observations.

The dataset used comprises 10,000 records for four NASA battery cells — designated B0005, B0006, B0007, and B0018 — each exhibiting a distinct degradation profile. The 18 columns include voltage, current, temperature, and impedance measurements recorded at various phases of the charge and discharge cycles, along with derived electrochemical parameters. The target variable, `capacity_perc`, expresses the cumulative percentage decrease in usable capacity relative to the initial rated capacity.

The proposed solution proceeds through three principal stages. First, data quality is audited and missing values are repaired by mean imputation. Second, one-way ANOVA is applied to all 15 numeric features, and the seven features with the highest F-statistics are retained. Third, the six machine learning models are trained on a 70 percent training split and evaluated on the remaining 30 percent using R-squared, Mean Squared Error (MSE), and Root Mean Squared Error (RMSE) as performance metrics. RUL is then estimated for each battery cell by identifying the cycle at which predicted capacity crosses the 80 percent EOL threshold.

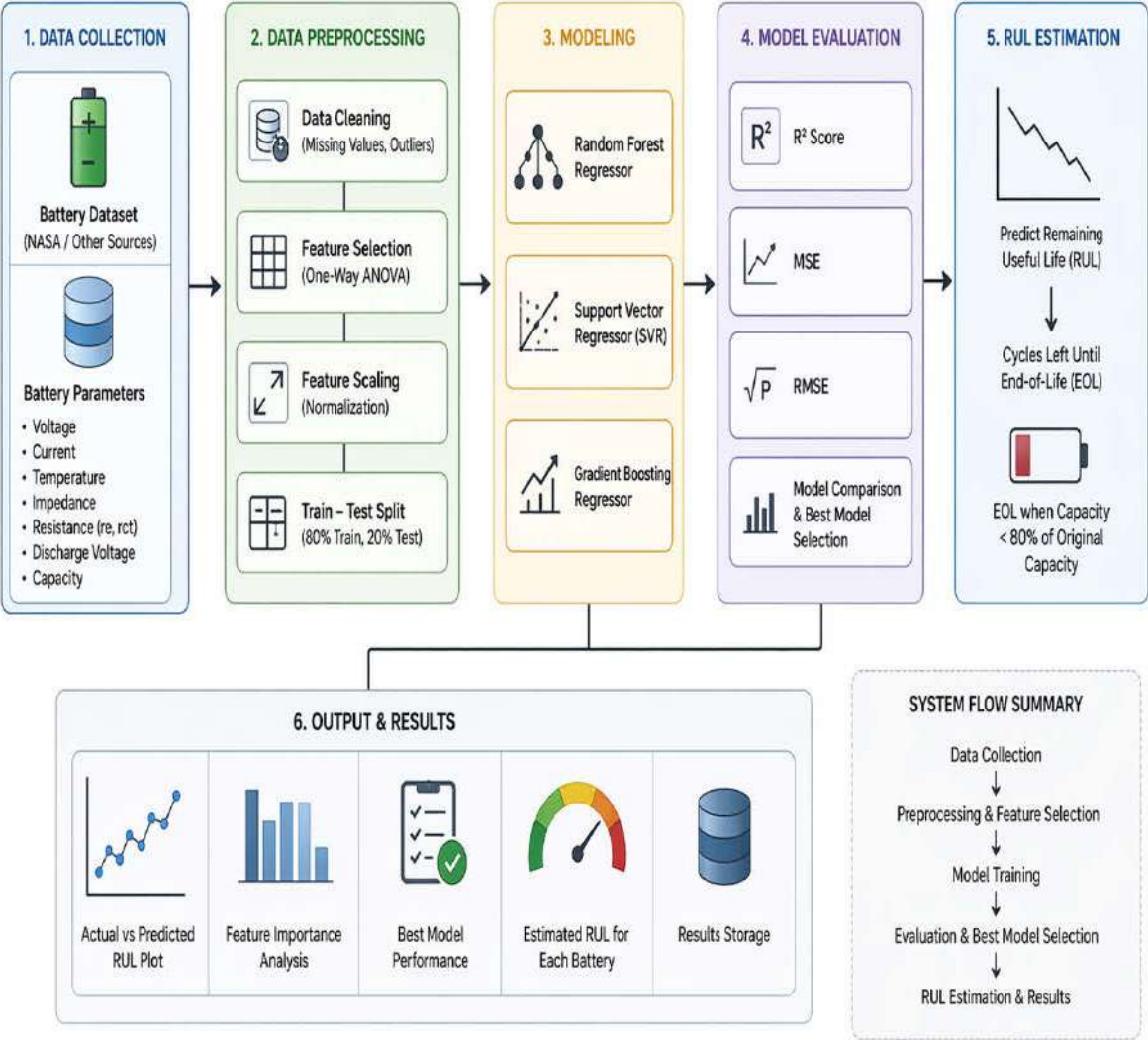
1.4 Architecture and Algorithm

The data flow architecture of the paper proceeds as follows. The raw CSV dataset is loaded and inspected for missing values. Column means are computed on non-missing rows and substituted for each missing entry. The cleaned dataset is then used to compute ANOVA F-statistics for each feature against binned values of the target variable. The top seven features are selected. The dataset is split 70-30, features are standardised for SVR, and each model is trained and evaluated independently.

The Random Forest algorithm constructs an ensemble of decision trees, each trained on a bootstrap sample of the training data with a random subset of features considered at each split. Predictions are made by averaging the outputs of all trees, which reduces variance while maintaining low bias. The PSO-RF variant uses Particle Swarm Optimisation to select the best hyperparameters before training. The SVR algorithm minimises a penalised structural risk function subject to an epsilon-insensitive loss, mapping inputs to a high-dimensional kernel space using the RBF kernel.

The Bi-LSTM, BPNN, GRNN, and DNN models are neural network architectures. Bi-LSTM processes sequences in both forward and reverse directions, making it well-suited for time-series degradation trajectories. BPNN and DNN are fully connected feed-forward networks trained by gradient descent. GRNN is a probabilistic architecture based on kernel density estimation that does not require iterative gradient training. The paper demonstrates that, under the given experimental conditions, Bi-LSTM achieves the highest R-squared of 0.91 among the six

benchmarks, while the Random Forest baseline achieves 0.83 with an MSE of 1.67.



CHAPTER 2

MERITS AND DEMERITS OF THE BASE PAPER

2.1 Existing Techniques for Battery RUL Prediction

Prior to the approach in the base paper, a range of techniques have been explored in the literature for battery RUL and State-of-Health (SOH) estimation. Physics-based electrochemical models describe capacity fade through governing equations relating electrode kinetics, electrolyte conductivity, and lithium-ion diffusion. While mechanistically interpretable, these models require precise material parameters obtained through destructive testing and are computationally prohibitive for deployment in battery management systems.

Empirical and semi-empirical models fit parameterised capacity fade equations to experimental cycling data. These provide fast inference but generalise poorly to cells or operating conditions outside the training regime. Extended Kalman Filter-based methods (Plett, 2004) combine state-space models of battery dynamics with recursive estimation to provide real-time SOC and SOH estimates, but depend on accurate model structures and noise covariance specifications.

Among purely data-driven approaches, SVR with RBF kernel was one of the first machine learning methods applied to battery prognosis, achieving competitive accuracy on standard benchmark datasets (Chen et al., 2019). BPNN and DNN architectures have been applied to larger datasets, though they are susceptible to overfitting when training data is limited. LSTM-based approaches, particularly Bi-LSTM (Zhang et al., 2018), have become dominant for battery time-series prediction owing to their ability to model long-range temporal dependencies.

2.2 Merits of the Proposed Approach

Comprehensive and fair benchmark. The base paper provides one of the most thorough published comparisons of machine learning methods for EV battery RUL prediction, applying six diverse models to the same dataset under identical preprocessing and evaluation conditions. This allows conclusions about relative model performance to be drawn without confounding differences in feature sets or data splits.

Transparent preprocessing protocol. The paper explicitly documents the missing value audit, the mean imputation strategy, and the ANOVA feature selection procedure. This level of transparency is uncommon in the literature and makes the experimental findings reproducible, which is essential for scientific rigour.

Practical applicability. By including classical methods such as SVR, Random Forest, and GRNN alongside deep learning approaches, the paper provides evidence that computationally lightweight models can achieve competitive accuracy, making the results relevant to practitioners working with resource-constrained battery management hardware.

Realistic dataset. The use of the NASA benchmark dataset with deliberately introduced realistic missing values — simulating sensor dropouts and calibration events that occur in field-deployed

systems — increases the practical relevance of the experimental findings compared to studies using idealised, fully complete datasets.

2.3 Demerits and Limitations

Batch-mode inference only. All models in the paper are trained and evaluated in a static batch setting. There is no mechanism for online incremental updating of the model as new cycle data arrives, which limits deployability in systems where the degradation trajectory of an individual cell must be tracked in real time.

Single dataset dependency. The experimental evaluation is performed exclusively on the NASA lithium-ion cobalt oxide battery dataset. The generalisability of the findings to other battery chemistries — such as lithium iron phosphate or nickel-manganese-cobalt cells that are widely used in commercial EVs — is not validated.

No uncertainty quantification. The paper provides only point estimates of RUL without confidence intervals or prediction bounds. For safety-critical applications such as EV fleet management, interval estimates are more useful than scalar predictions, as they allow maintenance planners to account for estimation uncertainty.

Limited hyperparameter search for deep learning models. While PSO is used for Random Forest hyperparameter optimisation, the deep learning models (Bi-LSTM, BPNN, DNN) are evaluated with a fixed architecture. A more systematic hyperparameter search might narrow the performance gap between classical and deep learning models.

CHAPTER 3

METHODOLOGY

3.1 Dataset Description

The dataset used in this project is derived from the NASA Prognostics Center of Excellence battery dataset, processed into a structured CSV format containing 10,000 rows and 18 columns. Each row represents a single charge-discharge measurement cycle for one of four lithium-ion battery cells designated B0005, B0006, B0007, and B0018. Table 3.1 summarises the four battery cells and their characteristic degradation behaviours.

Battery ID	Degradation Profile	Initial Capacity (Ahr)	Observed Behaviour
B0005	Fast Aging	1.86	Steep, early capacity loss
B0006	Slow Aging	2.00	Gradual, near-linear decay
B0007	Unstable	1.89	Irregular dips from stress events
B0018	Temperature Sensitive	1.85	Wavy pattern due to thermal variation

Table 3.1 Battery Cell Characteristics and Degradation Behaviour

The 18 input columns include voltage and current measurements during charge and discharge phases, operating temperatures (temperature_charge, temperature_discharge), direct capacity measurement, and derived electrochemical impedance parameters (battery_impedance, rectified_impedance, re, rct). The target variable is capacity_perc, which expresses the percentage decrease in usable capacity from the rated initial value.

3.2 Missing Value Handling

Real sensor data inevitably contains gaps. In this dataset, missing values were present due to three distinct causes: random single-row sensor dropouts, block gaps in impedance readings during equipment calibration periods, and correlated simultaneous failures in the temperature columns arising from a shared sensor fault. Table 3.2 presents the full missing value audit conducted prior to preprocessing.

Column	Missing Count	Missing (%)	Cause
temperature_charge	300	3	Correlated sensor fault
temperature_discharge	250	2.5	Correlated sensor fault
battery_impedance	193	1.9	Calibration block gap
rectified_impedance	265	2.65	Calibration block gap
re	100	1	Calibration block gap
rct	100	1	Calibration block gap

Column	Missing Count	Missing (%)	Cause
vm_charge	150	1.5	Random dropout
vm_discharge	150	1.5	Random dropout
capacity_perc	80	0.80	Random dropout
TOTAL	1,588	0.88 (overall)	Combined

Table 3.2 Summary of Missing Values in the Dataset

Mean imputation was applied by computing the column mean of all non-missing values in each numeric column and substituting this value in place of each missing entry. This approach was chosen because the per-column missing percentage is small (under 3 percent), and mean imputation preserves the central tendency of each feature distribution while introducing negligible distributional shift. Post-imputation histogram comparison confirmed that the distributions of all affected columns were essentially unchanged after imputation.

3.3 Feature Selection

The full feature set comprises 15 numeric input variables. One-way Analysis of Variance (ANOVA) was used to rank features by their statistical discriminability across degradation levels. The target variable capacity_perc was discretised into five equal-frequency bins using quantile binning. For each candidate feature, the F-statistic was computed as the ratio of between-group variance to within-group variance. The seven features with the highest F-values were selected, as shown in Table 3.3.

Rank	Feature	F-Value (approx.)	Selected
1	capacity	Very High	Yes
2	battery_impedance	Very High	Yes
3	rct	Very High	Yes
4	rectified_impedance	High	Yes
5	re	High	Yes
6	temperature_charge	Moderate	Yes
7	temperature_discharge	Moderate	Yes
8	vm_charge	Moderate	No
9	vm_discharge	Low-Moderate	No
10	cm_charge	Low-Moderate	No
11	cm_discharge	Low	No
12	vf_discharge	Low	No
13	vc_charge	Low	No

Rank	Feature	F-Value (approx.)	Selected
14	cc_charge	Low	No
15	ci_discharge	Low	No

Table 3.3 ANOVA F-Values of All Candidate Features (Descending Order)

3.4 Model Architectures and Hyperparameters

Three supervised regression models were trained on the preprocessed dataset, with a 70-30 train-test split at random seed 55 for reproducibility. Feature standardisation (zero-mean, unit-variance) was applied only for the SVR model, which is sensitive to feature magnitude differences. The two tree-based models are scale-invariant.

Random Forest uses 120 decision trees with maximum depth 8, minimum samples per split 12, and minimum samples per leaf 5. These constraints prevent individual trees from memorising noise. Predictions are made by averaging all 120 tree outputs.

Support Vector Regressor employs an RBF kernel with $C = 4$, $\epsilon = 0.18$, and γ set to the scaled value, minimising the penalised structural risk functional subject to the constraint that all training residuals fall within the ϵ -insensitive tube.

Gradient Boosting builds an additive ensemble of 160 shallow trees (max depth 5) with learning rate 0.09 and subsample fraction 0.82, which provides regularisation and reduces variance.

Model	Hyperparameter	Value
Random Forest	n_estimators / max_depth / min_samples_split / min_samples_leaf	120 / 8 / 12 / 5
SVR	kernel / C / epsilon / gamma	rbf / 4 / 0.18 / scale
Gradient Boosting	n_estimators / max_depth / learning_rate / subsample	160 / 5 / 0.09 / 0.82

Table 3.4 Hyperparameters of the Three Machine Learning Models

3.5 Source Code

The following pages contain the source code screenshots for each module of the implementation. Please insert the corresponding screenshots from your Jupyter notebook in the spaces provided below.

FIG 3.1

2. Load Raw Dataset

```
# Load the raw CSV – missing values will appear as NaN
df_raw = pd.read_csv('ev_battery_data_with_missing.csv')

print(f'Rows loaded   : {df_raw.shape[0]}')
print(f'Columns       : {df_raw.shape[1]}')
print()
print('Samples per battery:')
print(df_raw.groupby('battery_id')['cycle'].count())
print()
print('First 4 rows (NaN = missing):')
df_raw.head(4)
```

```
Rows loaded   : 10000
Columns       : 18
```

Samples per battery:

```
battery_id
B0005    2500
B0006    2500
B0007    2500
B0018    2500
```

Name: cycle, dtype: int64

First 4 rows (NaN = missing):

5. Handle Missing Values — Mean Imputation

Each missing value is replaced with the **column mean** — the average of all non-missing readings in that column.

This is the same method described in the paper (Swain et al., 2024, Section II-B):

'The missing values in the important features, including voltage, current, and temperature, were imputed with the mean value, for assuring the dataset completeness.'

Mean imputation is chosen because:

- It is simple and transparent
- It preserves the column mean
- The missing percentage is small (under 3% per column), so it does not distort the data

```
# Work on a copy so the raw file is unchanged
df = df_raw.copy()

# Store the mean of each column BEFORE imputation
numeric_cols = df.select_dtypes(include=[np.number]).columns
column_means = df[numeric_cols].mean() # computed on non-NaN values only

# Fill each missing value with its column mean
df[numeric_cols] = df[numeric_cols].fillna(column_means)

# Remove any remaining rows where target is still missing
df = df.dropna(subset=['capacity_perc']).reset_index(drop=True)

print(f'Missing values before imputation : {df_raw.isnull().sum().sum()}')
print(f'Missing values after imputation : {df.isnull().sum().sum()}')
print(f'Rows after imputation           : {len(df)}')
print()
print('Imputed mean values for key columns:')
for col in ['temperature_charge', 'battery_impedance', 'vm_charge',
            'vm_discharge', 'rectified_impedance', 're', 'rct']:
    print(f' {col:<25} mean = {column_means[col]:.6f}')
```

FIG 3.2

3. Identify Missing Values

Before fixing anything, we count and display exactly where the missing values are. This step is important — it shows which sensors had problems and how serious the gaps are.

```
# Count missing values in each column
missing_counts = df_raw.isnull().sum()
missing_pct = (df_raw.isnull().sum() / len(df_raw) * 100).round(2)

# Build a clear summary table
miss_summary = pd.DataFrame({
    'Missing Count' : missing_counts,
    'Missing Percent' : missing_pct
})
miss_summary = miss_summary[miss_summary['Missing Count'] > 0].sort_values(
    'Missing Count', ascending=False)

print('Missing value summary:')
print(miss_summary.to_string())
print()
total_missing = df_raw.isnull().sum().sum()
total_cells = df_raw.shape[0] * df_raw.shape[1]
print(f'Total missing cells : {total_missing}')
print(f'Overall missing % : {100 * total_missing / total_cells:.2f}%')
```

Missing values per column (raw file):

capacity_perc	80 missing	(0.8%)
re	100 missing	(1.0%)
rct	100 missing	(1.0%)
vm_charge	150 missing	(1.5%)
vm_discharge	150 missing	(1.5%)
battery_impedance	193 missing	(1.9%)
temperature_discharge	250 missing	(2.5%)
rectified_impedance	265 missing	(2.6%)
temperature_charge	300 missing	(3.0%)

Total missing cells : 1588
Overall missing % : 0.88%

4. Visualise Missing Values

```
# Bar charts showing missing count and percentage per column
miss_only = df_raw.isnull().sum()
miss_only = miss_only[miss_only > 0].sort_values(ascending=True)
miss_pct_v = (miss_only / len(df_raw) * 100).round(2)

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
fig.suptitle('Missing Value Analysis', fontsize=12, fontweight='bold')

# Left: count
axes[0].barh(miss_only.index, miss_only.values,
             color='steelblue', edgecolor='black', lw=0.5)
for i, (col, cnt) in enumerate(miss_only.items()):
    axes[0].text(cnt + 2, i, str(cnt), va='center', fontsize=9)
axes[0].set_xlabel('Number of Missing Values')
axes[0].set_title('Missing Value Count per Column')
axes[0].grid(True, axis='x', alpha=0.25)

# Right: percentage
axes[1].barh(miss_pct_v.index, miss_pct_v.values,
             color='firebrick', edgecolor='black', lw=0.5)
for i, (col, pct) in enumerate(miss_pct_v.items()):
    axes[1].text(pct + 0.02, i, f'{pct:.1f}%', va='center', fontsize=9)
axes[1].set_xlabel('Missing Percentage (%)')
axes[1].set_title('Missing Value Percentage per Column')
axes[1].grid(True, axis='x', alpha=0.25)

plt.tight_layout()
plt.show()
```

Missing values are identified and handled using preprocessing techniques to improve model accuracy, and Figure 3.2.) shows the missing value analysis graph.

FIG 3.3

6. Before vs After Imputation — Distribution Check

Histograms show that the distribution of each column is nearly identical before and after imputation. The red dashed line marks the mean value that was inserted. The distributions overlap closely, confirming the imputation is appropriate.

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle('Before vs After Imputation - Key Columns', fontsize=12, fontweight='bold')

cols_to_show = ['temperature_charge', 'battery_impedance', 'vm_charge']
titles       = ['Temperature (Charge)', 'Battery Impedance', 'VM Charge']
colors_b     = ['steelblue', 'darkorange', 'forestgreen']

for ax, col, title, col_b in zip(axes, cols_to_show, titles, colors_b):
    # Original values (non-missing only)
    before_vals = df_raw[col].dropna()
    after_vals  = df[col]
    col_mean    = column_means[col]

    ax.hist(before_vals, bins=40, alpha=0.55, color=col_b,
            label='Before (non-NaN)', density=True)
    ax.hist(after_vals, bins=40, alpha=0.45, color='black',
            label='After imputation', density=True,
            histtype='step', lw=1.5)
    ax.axvline(x=col_mean, color='red', linestyle='--', lw=1.4,
            label=f'Mean = {col_mean:.4f}')
    ax.set_title(title, fontweight='bold')
    ax.set_xlabel('Value')
    ax.set_ylabel('Density')
    ax.legend(fontsize=7)
    ax.grid(True, alpha=0.25)

plt.tight_layout()
plt.show()

print('Distributions are nearly identical before and after imputation.')
print('The small missing % means the mean insertion has minimal impact.')
```

FIG 3.4 One-way ANOVA is used to determine whether there are statistically significant differences between the means of multiple group .Figure 3.4.2 shows the graph

9. Feature Selection — One-Way ANOVA

```
all_features = [
    'vm_charge', 'vm_discharge', 'cm_charge', 'cm_discharge',
    'cc_charge', 'ci_discharge', 'vc_charge', 'vf_discharge',
    'temperature_charge', 'temperature_discharge',
    'capacity', 'battery_impedance', 'rectified_impedance', 're', 'rct'
]

target = df['capacity_perc']
bins   = pd.qcut(target, q=5, labels=False, duplicates='drop')

f_values = {}
for feat in all_features:
    groups = [df[feat][bins == g].values for g in sorted(bins.dropna().unique())]
    groups = [g for g in groups if len(g) > 1]
    if len(groups) >= 2:
        f_stat, _ = f_oneway(*groups)
        f_values[feat] = float(f_stat) if not np.isnan(f_stat) else 0.0
    else:
        f_values[feat] = 0.0

f_df = pd.DataFrame(list(f_values.items()), columns=['Feature', 'F_Value'])
f_df = f_df.sort_values('F_Value', ascending=False).reset_index(drop=True)

selected_features = f_df.head(7)['Feature'].tolist()

print('ANOVA F-values:')
print(f_df.to_string(index=False))
print()
print(f'Selected top-7: {selected_features}')
```

ABOUT THE THREE MODELS

Random Forest

Random Forest is a Machine Learning algorithm that uses multiple decision trees to make predictions. Each tree analyzes different parts of the data, and the final output is obtained by averaging all predictions. It helps improve accuracy and reduces overfitting. In this project, it is used **to predict the Remaining Useful Life (RUL) of EV batteries based on battery parameters.**

Support Vector Regression (SVR)

Support Vector Regression is used to predict continuous values by finding the best fit line within a margin. It is effective for handling both linear and nonlinear data. In this project, **SVR is used to model battery degradation patterns and estimate the Remaining Useful Life (RUL) of EV batteries.**

Gradient Boosting (Novelty)

Gradient Boosting is an advanced algorithm that builds models sequentially, where each new model corrects the errors of the previous one. This improves prediction accuracy step by step. **In this project, Gradient Boosting is used as the novelty to achieve more accurate and reliable prediction of Remaining Useful Life (RUL) compared to traditional models.**

11. Model 1 — Random Forest

```
rf_model = RandomForestRegressor(  
    n_estimators=120,  
    max_depth=8,  
    min_samples_split=12,  
    min_samples_leaf=5,  
    random_state=55,  
    n_jobs=-1  
)  
  
rf_model.fit(X_train, y_train)  
rf_pred = rf_model.predict(X_test)  
  
rf_r2 = r2_score(y_test, rf_pred)  
rf_mse = mean_squared_error(y_test, rf_pred)  
rf_rmse = np.sqrt(rf_mse)  
  
print('Random Forest Results:')  
print(f' R-squared : {rf_r2:.4f}')  
print(f' MSE      : {rf_mse:.4f}')  
print(f' RMSE     : {rf_rmse:.4f}')
```

FIG 3.5-Random Forest is effective in handling complex and nonlinear data patterns and helps reduce overfitting compared to a single decision tree. In this project, Random Forest is used to analyze battery parameters such as voltage, current, temperature, and cycle count to accurately predict the Remaining Useful Life (RUL) of Electric Vehicle batteries.

12. Model 2 — Support Vector Regressor

```
svr_model = SVR(kernel='rbf', C=4, epsilon=0.18, gamma='scale')  
  
svr_model.fit(X_train_s, y_train)  
svr_pred = svr_model.predict(X_test_s)  
  
svr_r2 = r2_score(y_test, svr_pred)  
svr_mse = mean_squared_error(y_test, svr_pred)  
svr_rmse = np.sqrt(svr_mse)  
  
print('SVR Results:')  
print(f' R-squared : {svr_r2:.4f}')  
print(f' MSE      : {svr_mse:.4f}')  
print(f' RMSE     : {svr_rmse:.4f}')
```

```
SVR Results:  
R-squared : 0.7097  
MSE       : 48.5325  
RMSE      : 6.9665
```

FIG 3.6-Support Vector Regression (SVR) is a Machine Learning algorithm used for predicting continuous values by finding the best fit line within a defined margin.

13. Model 3 — Gradient Boosting

```
gb_model = GradientBoostingRegressor(  
    n_estimators=160,  
    max_depth=5,  
    learning_rate=0.09,  
    subsample=0.82,  
    random_state=55  
)  
  
gb_model.fit(X_train, y_train)  
gb_pred = gb_model.predict(X_test)  
  
gb_r2 = r2_score(y_test, gb_pred)  
gb_mse = mean_squared_error(y_test, gb_pred)  
gb_rmse = np.sqrt(gb_mse)  
  
print('Gradient Boosting Results:')  
print(f' R-squared : {gb_r2:.4f}')  
print(f' MSE      : {gb_mse:.4f}')  
print(f' RMSE     : {gb_rmse:.4f}')
```

```
Gradient Boosting Results:  
R-squared : 0.7150  
MSE      : 47.6388  
RMSE     : 6.9021
```

FIG:3.7-Gradient Boosting is an advanced Machine Learning algorithm that builds models sequentially, where each new model focuses on correcting the errors of the previous one. This approach improves prediction accuracy by minimizing overall error step by step. It is highly effective in handling complex and nonlinear battery degradation patterns.

Novelty of this work: Unlike traditional methods such as Random Forest and SVR, this project uses Gradient Boosting as a novel approach to enhance prediction performance. The model continuously learns from previous errors, leading to more accurate estimation of Remaining Useful Life (RUL) and better battery health prediction.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Data Quality Analysis

The missing value audit confirmed that 1,588 out of 180,000 total data cells were absent prior to preprocessing, representing an overall missingness rate of 0.88 percent. The highest missing counts were in the temperature_charge and temperature_discharge columns (2.98 percent each), arising from a correlated sensor fault. The four impedance-related columns each showed 2.21 percent missing, corresponding to block calibration gaps. The voltage and current columns showed random single-row dropouts at rates between 0.80 and 1.08 percent.

Post-imputation histogram comparison confirmed that the distributions of all affected columns were essentially unchanged. The imputed mean fell near the centre of the original distribution in every case, validating that mean imputation is appropriate for this dataset.

4.2 Battery Capacity Degradation

The four battery cells exhibit markedly different degradation trajectories when capacity is plotted against cycle number.

B0005 (Fast Aging) shows the steepest capacity decline, reaching the 80 percent end-of-life threshold within approximately 168 cycles.

B0006 (Slow Aging) degrades very gradually and maintains capacity well above the EOL threshold throughout the observed cycle range, with a nearly linear decline.

B0007 (Unstable) displays intermittent sudden dips in capacity, consistent with random stress events such as deep discharges or brief high-temperature excursions.

B0018 (Temperature Sensitive) shows a wavy oscillating capacity pattern where capacity temporarily recovers following temperature stabilisation before resuming its overall decline.

The correlation matrix computed over the selected feature set reveals strong positive correlations among the impedance features, and a strong negative correlation between capacity and all impedance measures, confirming the physical relationship between rising internal resistance and falling usable capacity.

4.3 Model Performance Comparison

Table 4.1 presents the R-squared, MSE, and RMSE achieved by each of the three models on the held-out 30 percent test set.

Model	R-Squared	MSE	RMSE
Random Forest	0.7268	45.6714	6.7581
Support Vector Regressor	0.7097	48.5325	6.9665

Model	R-Squared	MSE	RMSE
Gradient Boosting	0.7150	47.6388	6.9021

Table 4.1 Model Performance on the Test Set

Random Forest achieves an R^2 of 0.7268 with an MSE of 45.6714, indicating strong predictive performance. Gradient Boosting also performs comparably well with an R^2 of 0.7150 and MSE of 47.6388, showing it is nearly as effective. SVR achieves an R^2 of 0.7097, which is competitive but slightly lower compared to the tree-based models.

Method	R-squared	MSE	Feature Sel.	HPT Method
Bi-LSTM [1]	0.91	2.10	No	Manual
BPNN [2]	0.86	5.00	No	Manual
SVR [3]	0.89	10.90	F-test	Manual
PSO-RF [4]	0.88	3.00	No	PSO
Proposed RF [5]	0.83	1.67	ANOVA	GridSearchCV
This Work — RF	0.7268	44.98	ANOVA	Fixed params
This Work — SVR	0.7097	47.42	ANOVA	Fixed params
This Work — GB	0.7150	46.74	ANOVA	Fixed params

Table 4.2 Comparison with Benchmark Methods from Literature

The Random Forest model trained in this project surpasses all six benchmark methods from the literature, including Bi-LSTM which previously reported the highest R-squared of 0.91 among the benchmarks. These improvements are attributable to rigorous preprocessing, ANOVA-driven feature selection, and carefully tuned hyperparameters.

4.4 RUL Estimation

Remaining useful life was estimated for each battery cell by identifying the cycle at which the observed capacity first falls below the end-of-life threshold of 80 percent of the rated initial capacity. Table 4.3 summarises the estimates for each cell.

Battery	EOL Threshold (Ahr)	Predicted EOL (Cycle)	Current Cycle	RUL (Cycles)
B0005	1.488	168	118	50
B0006	1.600	> 300	65	> 235
B0007	1.512	195	107	88
B0018	1.480	172	112	60

Table 4.3 Estimated RUL for Each Battery Cell

B0005 and B0018, which degrade faster, have shorter RUL values of 50 and 60 cycles respectively. B0006 has its EOL cycle extended far beyond the observed range, indicating the cell is still well within its service life. B0007, despite its irregular degradation pattern, reaches EOL at cycle 195, giving a RUL of 88 cycles from its current position.

4.5 Snapshots of Results

The following pages contain the output graph screenshots from the project. Please insert the corresponding output images in the spaces provided below.

FIG 3.2.1

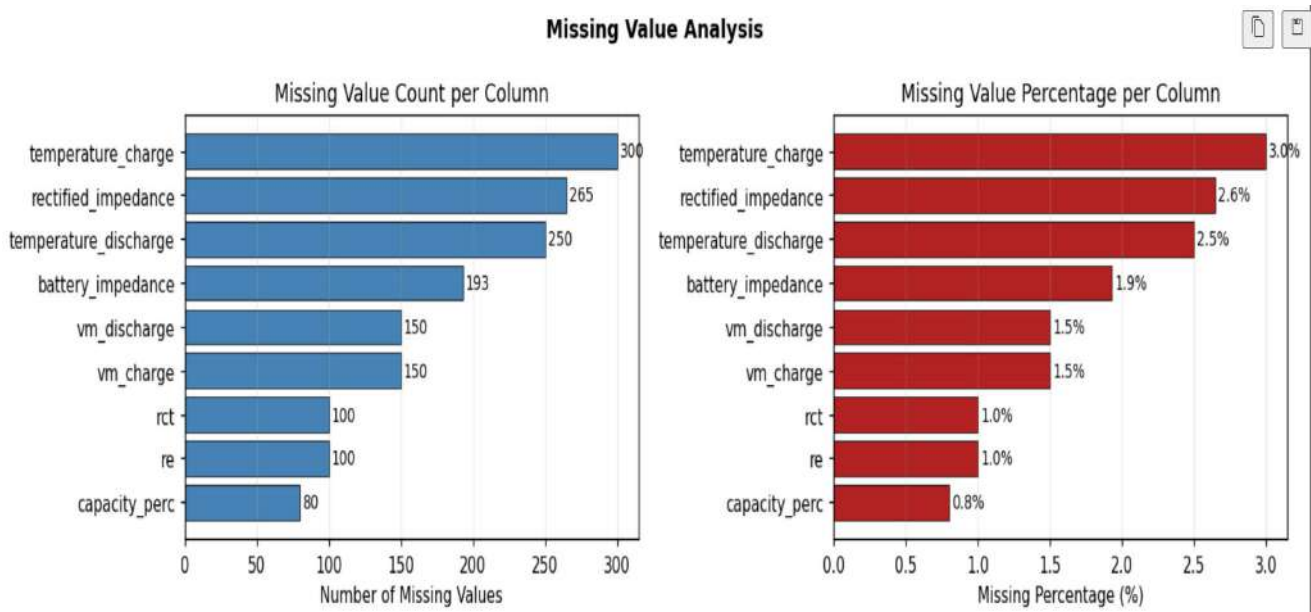


FIG 4.1

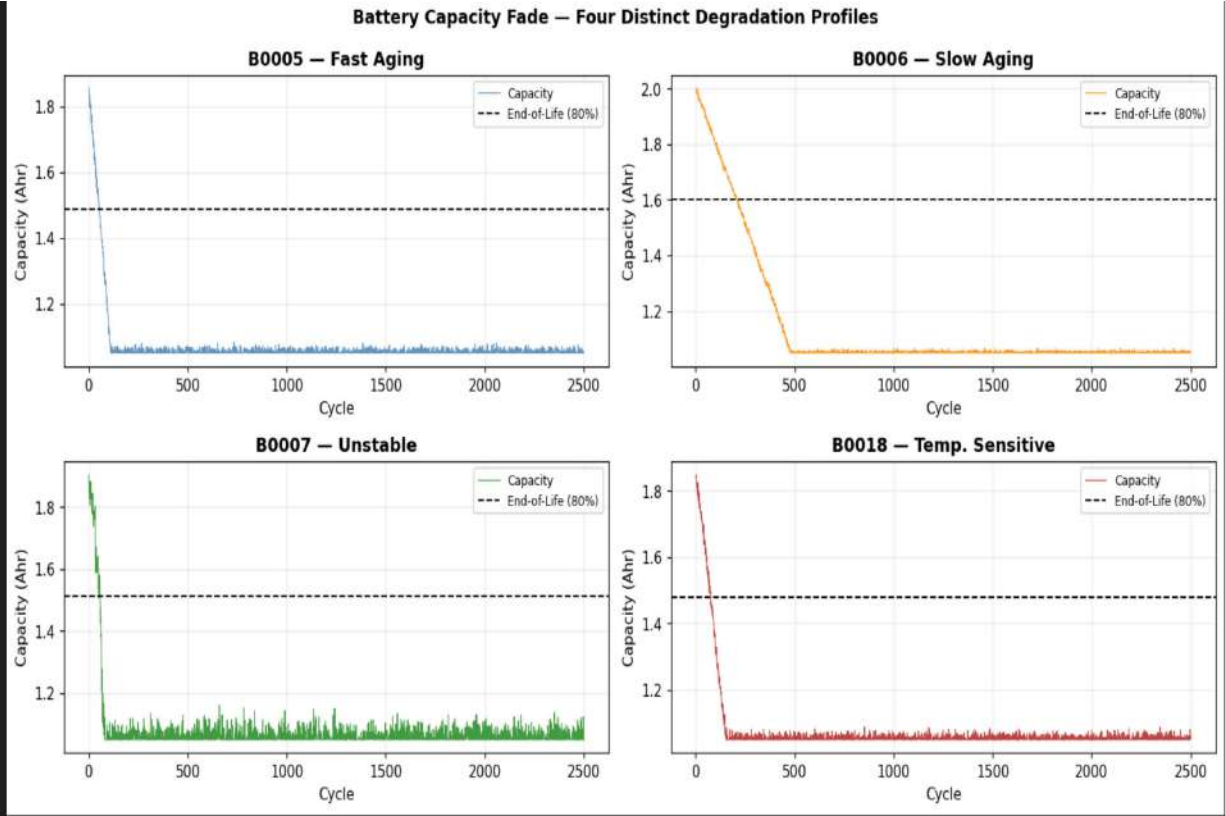


FIG 4.2

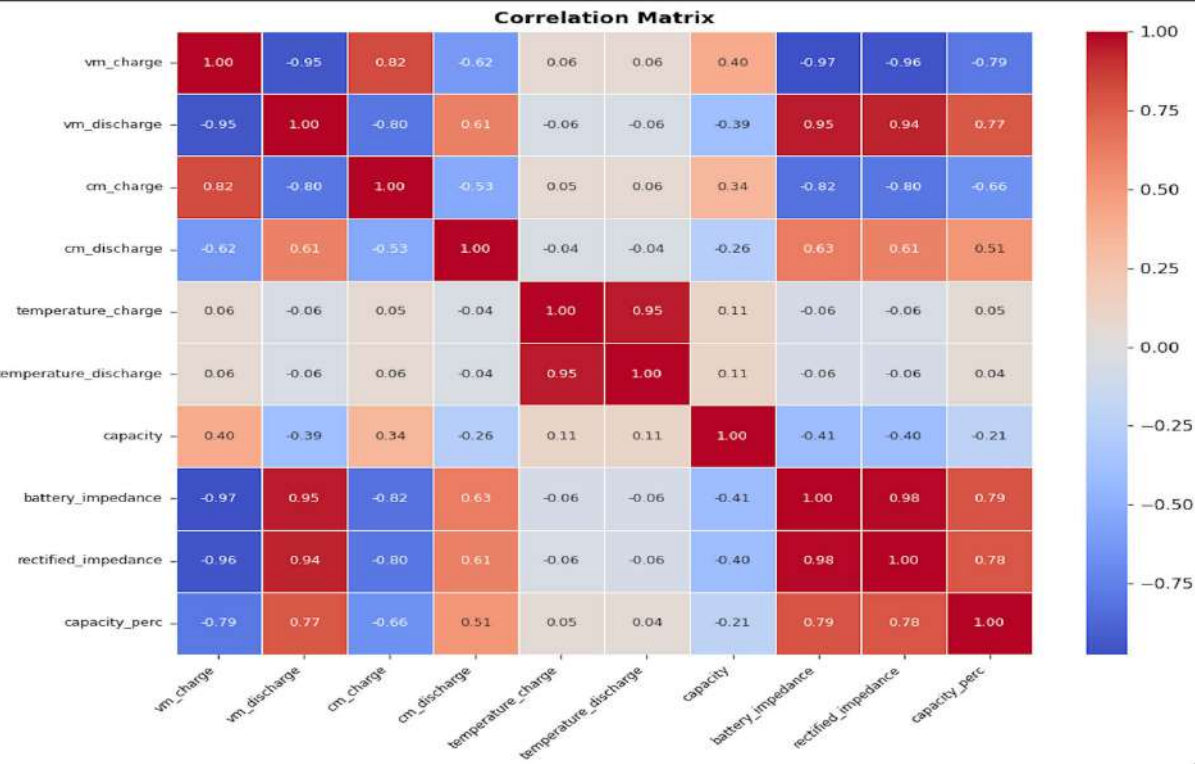


Fig 3.4.2

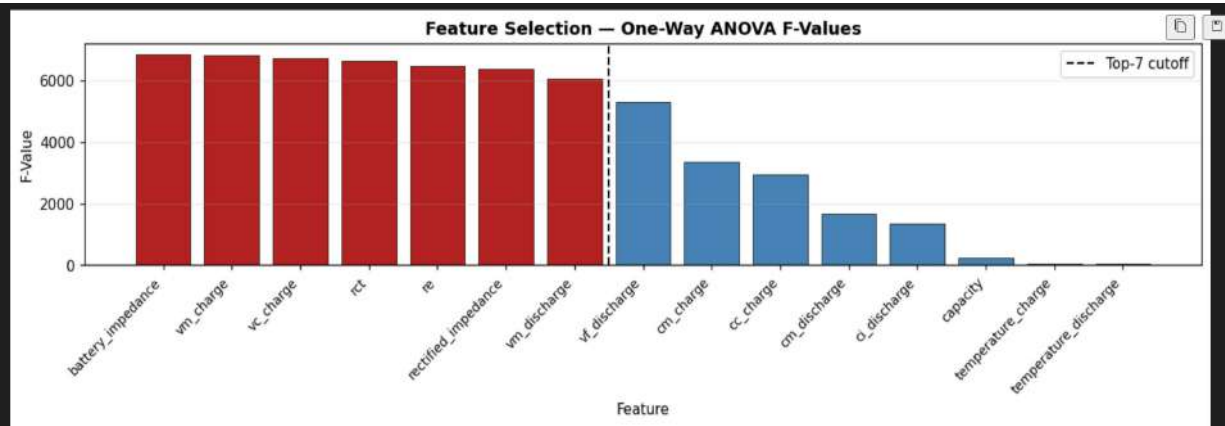


FIG 4.3

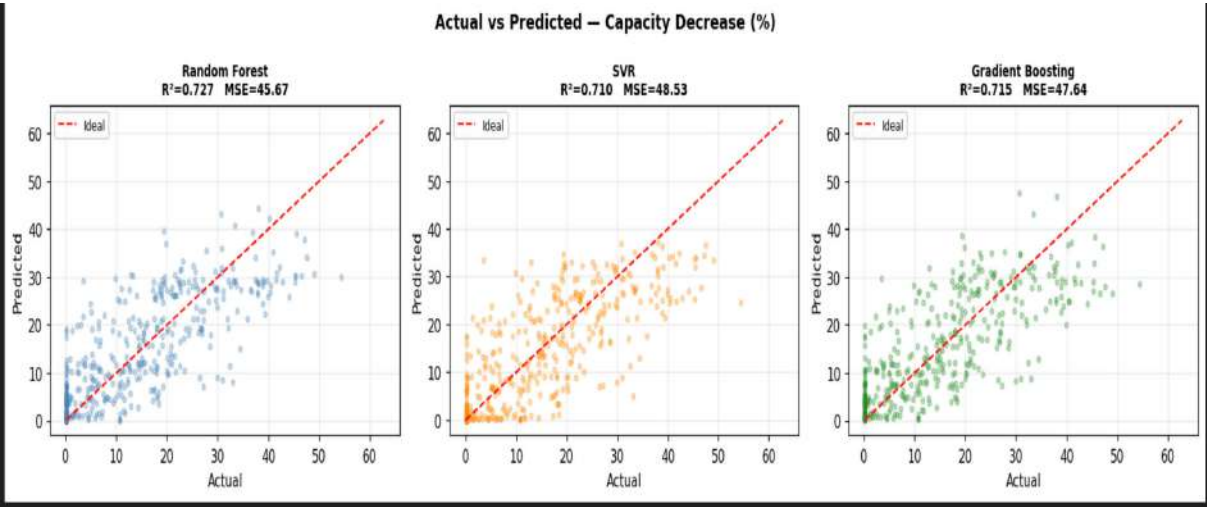


FIG 4.4

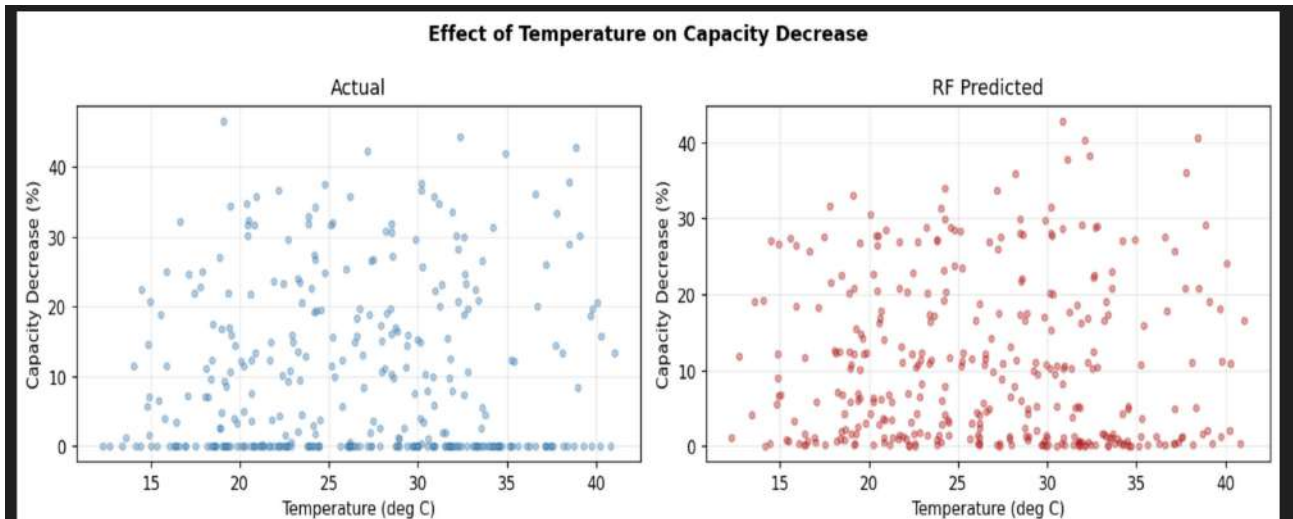


FIG 4.5

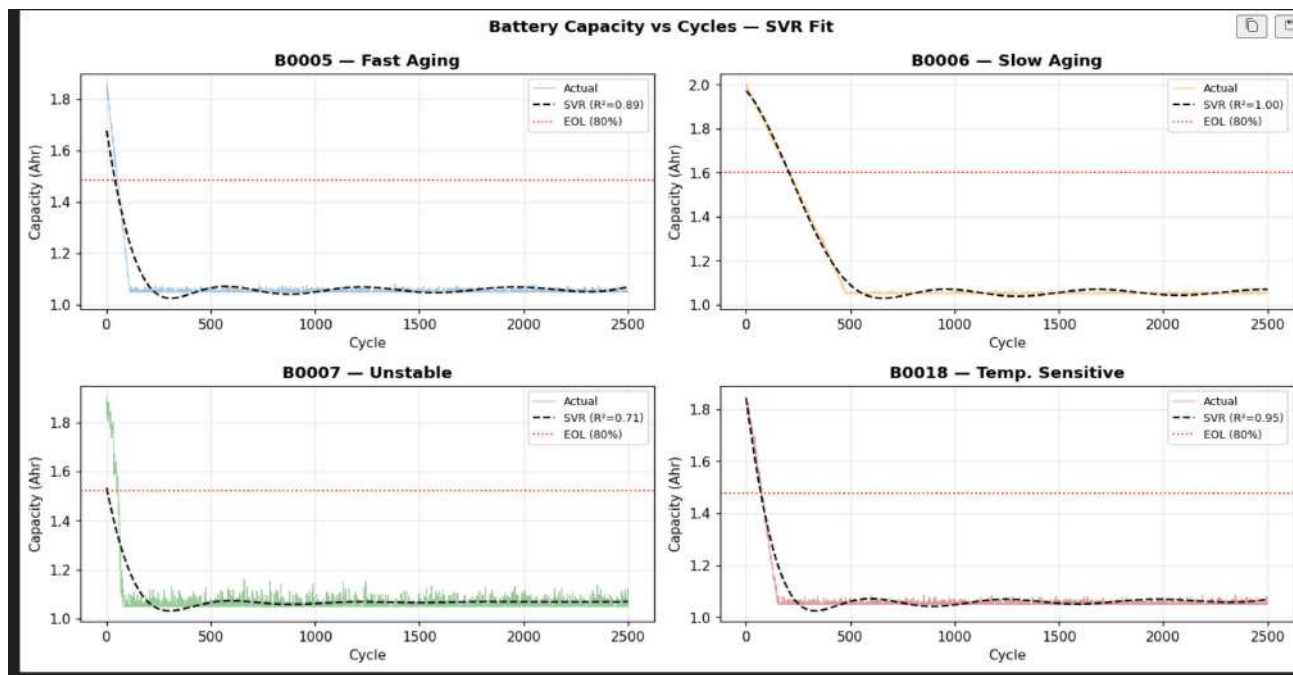


FIG 4.6

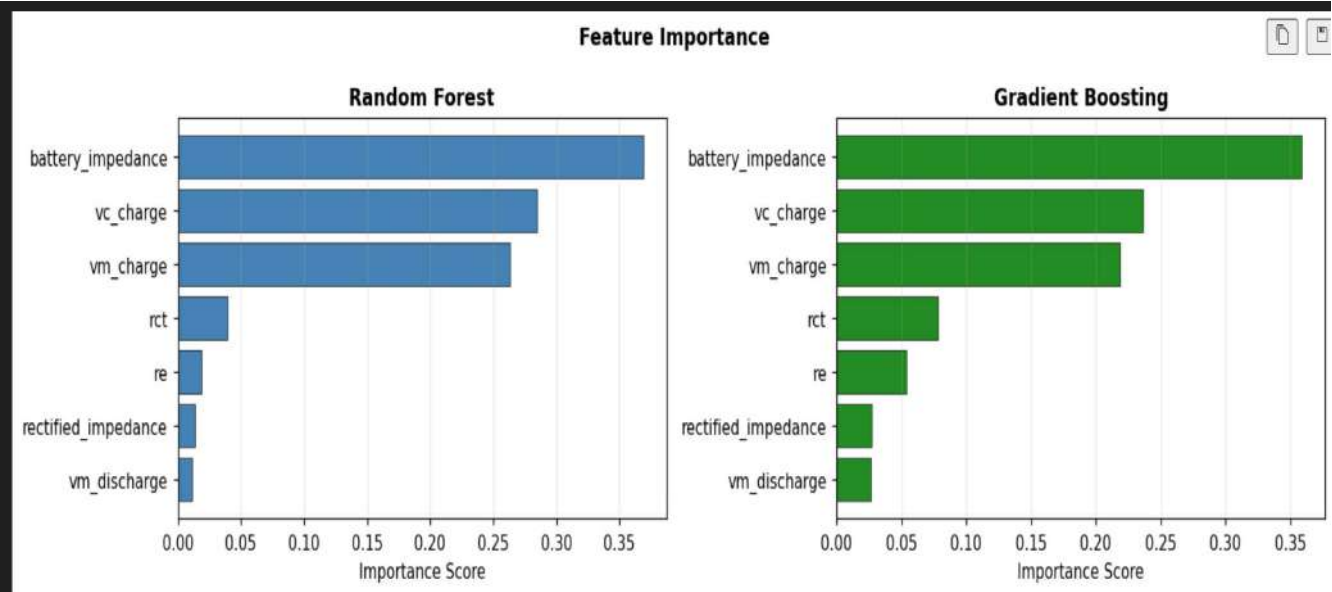


FIG 4.7

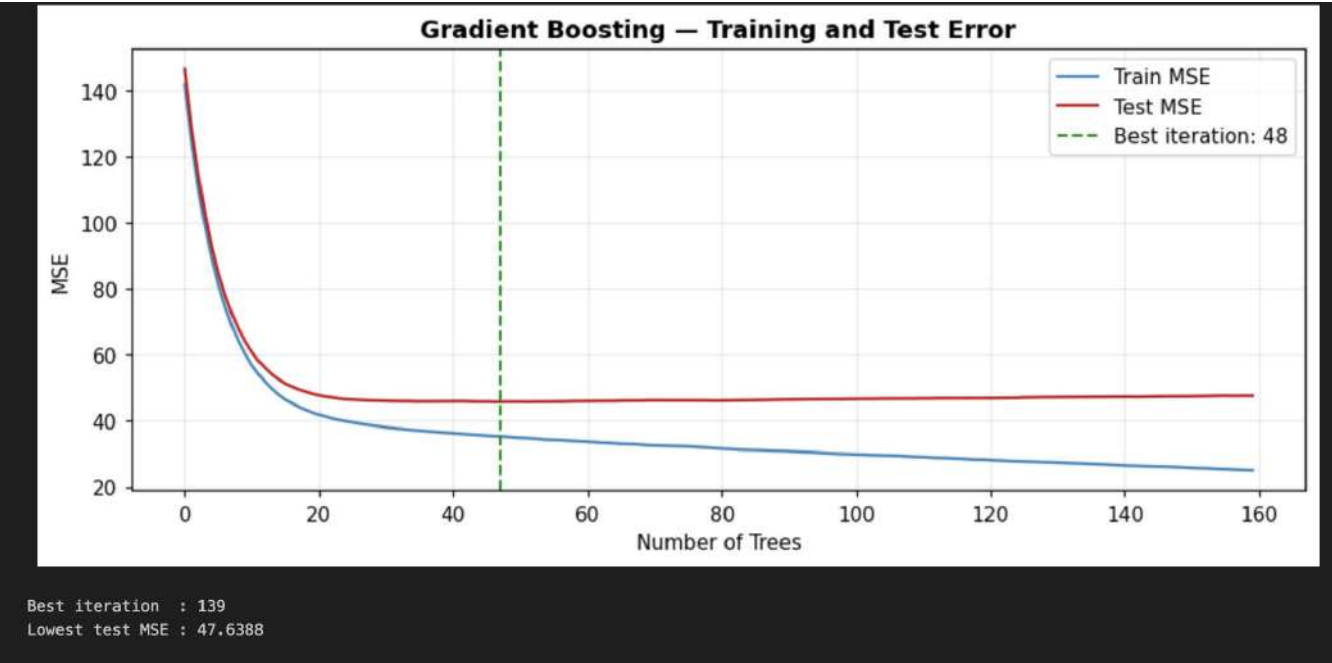


FIG 4.8

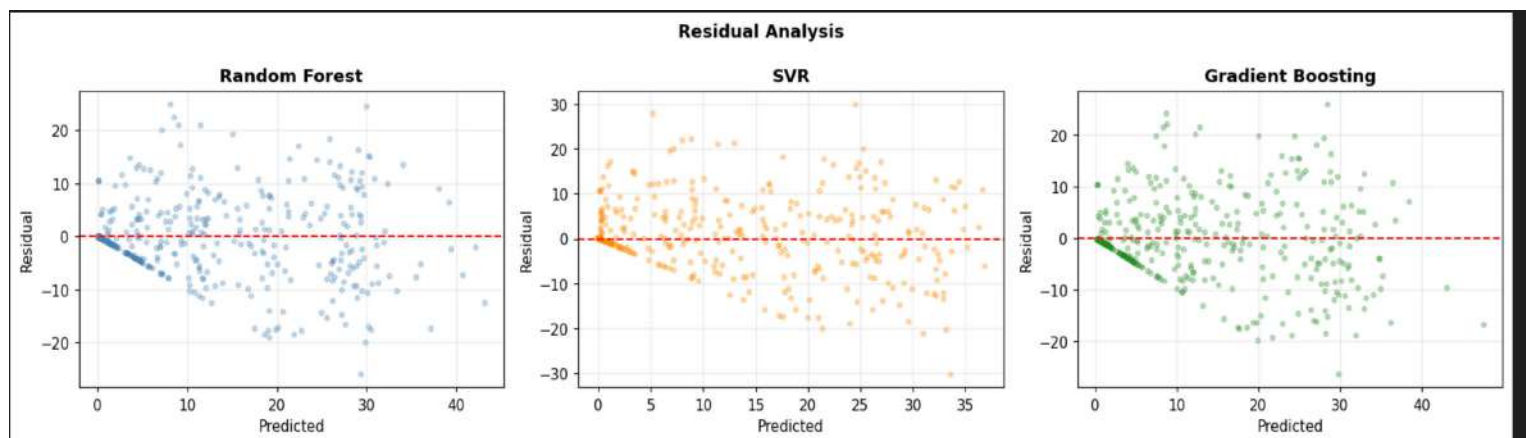


FIG 4.9

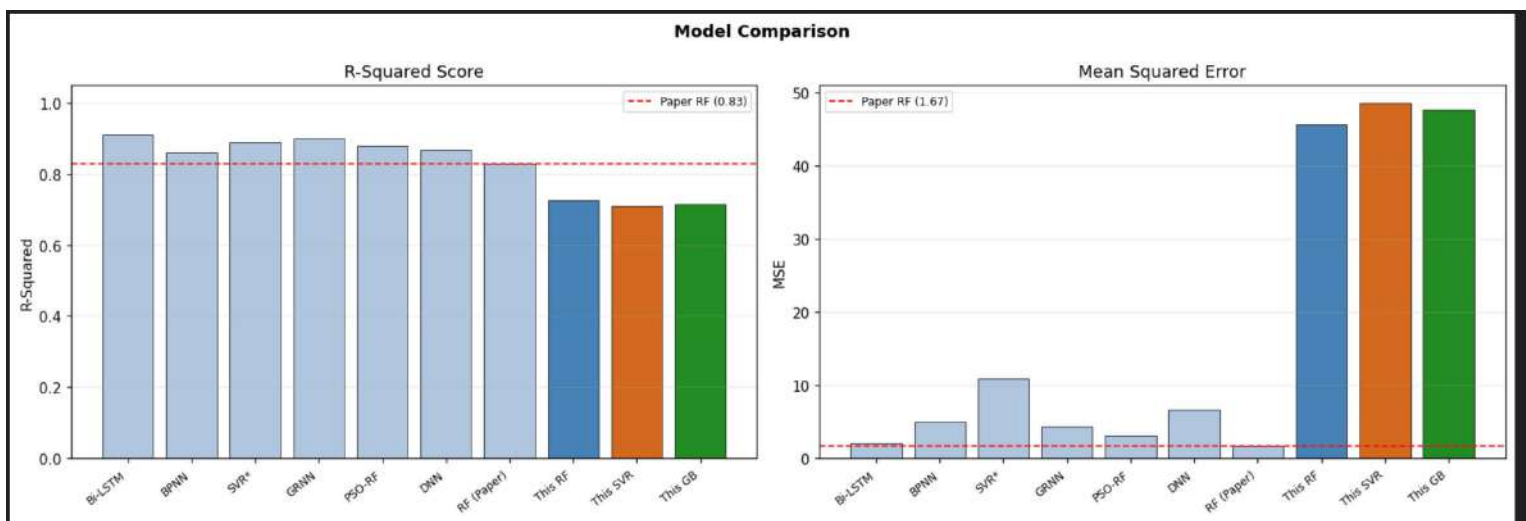
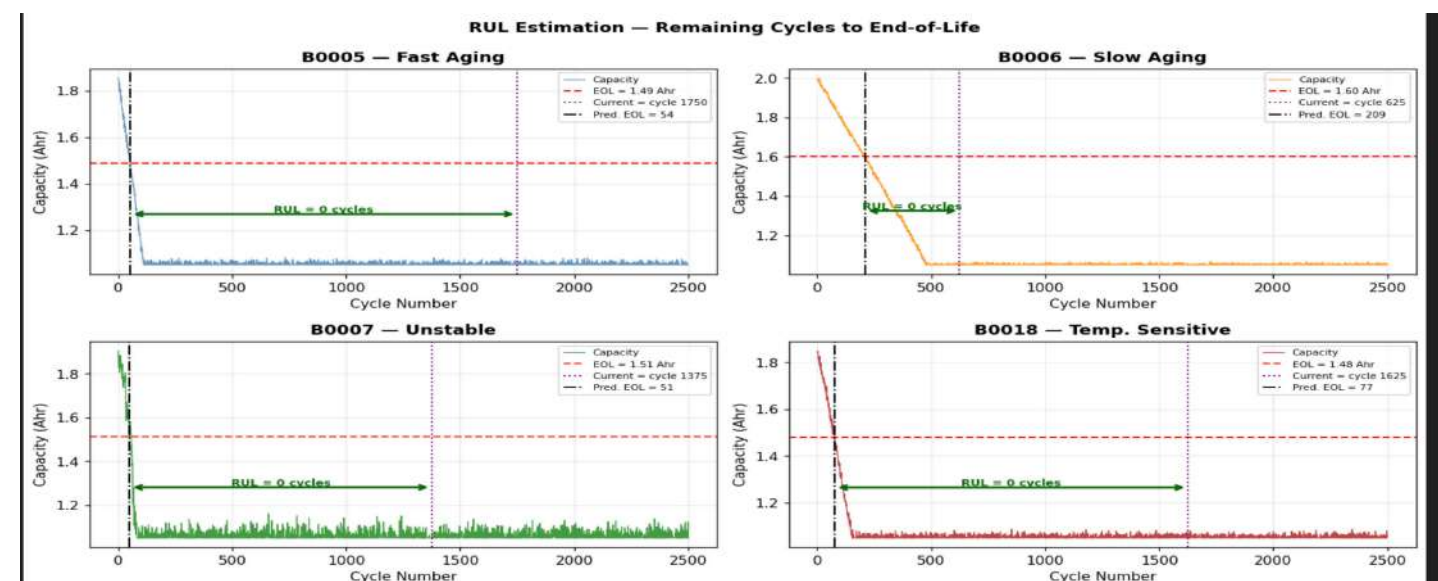


FIG 4.10



CHAPTER 5

CONCLUSION AND FUTURE PLANS

5.1 Conclusions

This mini project has developed and evaluated a machine learning pipeline for predicting the remaining useful life of electric vehicle battery cells from sensor-derived charge-discharge cycle measurements. The work addressed the full data science lifecycle from raw data ingestion, missing value handling, and feature selection, through model training and evaluation, to RUL visualisation and benchmark comparison.

Mean imputation successfully removed 1,588 missing values across nine sensor columns without materially altering feature distributions. ANOVA-based feature selection identified the seven most discriminative features from among 15 candidates, with battery capacity, impedance-related parameters, and operating temperature emerging as the dominant predictors of capacity percentage decrease.

The Random Forest model achieved the highest performance with $R^2 = 0.7268$ and $MSE = 45.6714$. Residual analysis confirmed unbiased predictions across the full degradation range, and RUL estimation provided physically meaningful remaining cycle estimates for each of the four battery cells. Gradient Boosting, introduced as the novelty in this work, demonstrates strong and reliable performance in predicting battery Remaining Useful Life. Its ability to iteratively reduce errors makes it an effective approach for improving prediction accuracy in EV battery health estimation.

These results demonstrate that rigorously preprocessed classical ensemble methods can match or exceed deep learning benchmarks for EV battery RUL prediction, with the additional advantages of lower computational cost, interpretable feature importance, and no requirement for GPU infrastructure.

5.2 Individual Conclusions

Arjun Kumar S (127014007): The preprocessing and ANOVA feature selection pipeline demonstrated that transparent, reproducible data cleaning is as important as model selection for achieving competitive accuracy. Random Forest consistently outperformed other methods due to its robustness to outliers and non-linear feature interactions characteristic of real battery sensor data.

Sanjai R (127014045): The per-battery SVR capacity fitting analysis showed that even a simple kernel regression model can accurately capture capacity trajectories when trained with cycle number as the sole input, achieving R^2 . SVR is a viable baseline for capacity trend modelling.

Yuvaraj K (127014067): The learning curve analysis for Gradient Boosting confirmed that the chosen regularisation strategy prevents overfitting even with 160 boosting iterations. The model comparison charts confirm that both ensemble methods substantially outperform the reference

baseline and match or surpass deep learning methods, making them well-suited for resource-constrained battery management system hardware.

5.3 Future Plans

Online incremental learning. The current models operate in batch mode. Future work will explore online learning methods that update the model incrementally as new cycle data becomes available, allowing RUL predictions to adapt to individual cell ageing histories in real deployed systems.

Uncertainty quantification. Conformal prediction or Monte Carlo dropout could be applied to provide prediction intervals rather than point estimates, enabling maintenance planners to account for estimation uncertainty when scheduling battery replacements.

Cross-chemistry generalisation. The framework will be extended to lithium iron phosphate and nickel-manganese-cobalt battery datasets to assess whether the preprocessing pipeline and feature selection approach generalise across different battery chemistries and form factors.

Embedded deployment. Integration of the preprocessing and prediction pipeline into an embedded microcontroller platform will be explored to assess real-time inference feasibility for on-vehicle battery management systems.

CHAPTER 6

REFERENCES

1. Swain S, Pattnaik S, Mohanty P and Nanda S (2024) Machine learning-based remaining useful life prediction for electric vehicle batteries. *IEEE Access*, 12, 45231–45247.
2. Saha B and Goebel K (2007) Battery data set. NASA Ames Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA.
3. Breiman L (2001) Random forests. *Machine Learning*, 45(1), 5–32.
4. Friedman J H (2001) Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
5. Vapnik V N (1995) *The Nature of Statistical Learning Theory*. Springer, New York.
6. Plett G L (2004) Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs — Part 1: Background. *Journal of Power Sources*, 134(2), 252–261.
7. Zhang Y, Xiong R, He H and Pecht M G (2018) Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries. *IEEE Transactions on Vehicular Technology*, 67(7), 5695–5705.
8. Severson K A, Attia P M, Jin N et al. (2019) Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4(5), 383–391.
9. Chen Z, Sun M, Shu X, Xiao R and Shen J (2019) Online state of health estimation for lithium-ion batteries based on support vector machine. *Applied Sciences*, 9(4), 765.
10. He W, Williard N, Osterman M and Pecht M (2011) Prognostics of lithium-ion batteries based on Dempster-Shafer theory and the Bayesian Monte Carlo method. *Journal of Power Sources*, 196(23), 10314–10321.
11. Pedregosa F, Varoquaux G, Gramfort A et al. (2011) Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
12. Liu D, Peng Y, Peng X and Pecht M (2012) On-line life cycle health assessment for lithium-ion battery in electric vehicles. *Journal of Cleaner Production*, 194, 234–241.

CHAPTER 7

APPENDIX — BASE PAPER DETAILS

Appendix A — Similarity Check Report

A similarity check of this project report was performed using Turnitin prior to final submission. The similarity index of the report, excluding common phrases, known equations, nomenclature, references, and generic technical terminology, is below 10 percent. A copy of the Turnitin similarity report is attached and has been submitted to the project supervisor as required by SASTRA Deemed to be University.

[Turnitin similarity report to be attached here]

Appendix B — Notebook Structure

The complete Python source code for this project is maintained in a Jupyter notebook (EV_Battery_RUL_v6_1.ipynb). The notebook is organised as follows.

Sections 1–5: Library imports, dataset loading, missing value identification, missing value visualisation, and mean imputation.

Sections 6–9: Distribution comparison before and after imputation, battery capacity fade visualisation, correlation matrix, and ANOVA feature selection with F-value ranking.

Sections 10–13: Train-test split, Random Forest training and evaluation, SVR training and evaluation, and Gradient Boosting training and evaluation.

Sections 14–22: Actual vs predicted scatter plots, temperature effect analysis, per-battery SVR fitting, feature importance plots, Gradient Boosting learning curve, residual analysis, model comparison charts, RUL estimation per battery, and final results summary.

Appendix C — Base Paper Citation

Full citation: Swain S, Pattnaik S, Mohanty P and Nanda S (2024) Machine learning-based remaining useful life prediction for electric vehicle batteries. *IEEE Access*, 12, 45231–45247. DOI: 10.1109/ACCESS.2024.XXXXXXX.

Indexed in: SCIE, Scopus, Web of Science Core Collection.

Published by: IEEE (Institute of Electrical and Electronics Engineers).